

Guia das Ferramentas de Cibersegurança

22 ferramentas explicadas em linguagem simples

Luís Soares · ciberacaro

2026

Conteúdo

Introdução	3
1. Análise de Sites	4
Verificador de Cabeçalhos de Segurança — <code>check_headers.py</code>	4
Verificador de Cookies — <code>cookie_check.py</code>	5
Verificador de Partilha entre Sites (CORS) — <code>cors_check.py</code>	5
Verificador de Métodos HTTP — <code>http_methods.py</code>	6
Leitor de robots.txt e Sitemap — <code>robots_check.py</code>	6
Comparador de Cabeçalhos ao Longo do Tempo — <code>header_diff.py</code>	7
Identificador de Tecnologias — <code>tech_fingerprint.py</code>	7
2. DNS e Rede	9
Consultor de Registos DNS — <code>dns_records.py</code>	9
Descobridor de Subdomínios — <code>subfinder.py</code>	9
Consultor de Registo de Domínio (WHOIS) — <code>whois_check.py</code>	10
Máquina do Tempo para Sites — <code>wayback_check.py</code>	11
3. Certificados e Ligações Seguras	12
Inspetor de Certificado e TLS — <code>tls_inspect.py</code>	12
4. Autenticação e Hashes	13
Analisador de Tokens JWT — <code>jwt_inspect.py</code>	13
Identificador de Hashes — <code>hashid.py</code>	13
5. Palavras-passe e Vulnerabilidades	15
Verificador de Força de Palavra-passe — <code>password_strength.py</code>	15
Detetor de Segredos no Código — <code>secrets_scan.py</code>	15
Consultor de Vulnerabilidades Conhecidas (CVE) — <code>cve_lookup.py</code>	16
6. Criptografia e CTF	17
Descodificador Automático — <code>multidecode.py</code>	17
Quebrador de Cifra XOR — <code>xor_crack.py</code>	17
7. Orquestração	19
Reconhecimento Automático Completo — <code>recon.py</code>	19
8. HackTheBox	20
Estatísticas do HackTheBox — <code>htb_stats.py</code>	20

9. Portfolio	21
Gerador de Writeups — <code>new_writeup.py</code>	21
Glossário Rápido	22

Introdução

Este guia explica 22 ferramentas de linha de comandos criadas para ajudar a analisar a segurança de sites, servidores e domínios. Estão organizadas por tema e cada uma é explicada em linguagem simples — sem pressupostos de conhecimento técnico avançado.

O que precisas para usar estas ferramentas:

- **Python 3.8 ou superior** instalado no teu computador (gratuito, disponível em python.org)
- Um **terminal** (no Mac: Terminal ou iTerm; no Windows: PowerShell ou WSL; no Linux: qualquer terminal)
- As ferramentas estão na pasta `tools/` do repositório — não precisas instalar nada extra

Como correr qualquer ferramenta:

```
python3 tools/nome-da-ferramenta.py --help
```

O parâmetro `--help` mostra todas as opções disponíveis. Todas as ferramentas aceitam `--lang pt` para mostrar o resultado em português.

Códigos de resultado:

Código	Significado
0	Tudo bem — nenhum problema encontrado
1	Problemas encontrados (ver relatório)
2	Erro de utilização (ex: URL mal escrito)
3	Erro de rede (site inacessível)

1. Análise de Sites

Ferramentas que analisam sites a partir do exterior — como um visitante normal faria, mas com mais atenção aos detalhes de segurança.

Verificador de Cabeçalhos de Segurança — `check_headers.py`

Para que serve: Quando acedes a um site, o servidor envia informações invisíveis chamadas “cabeçalhos”. Alguns desses cabeçalhos protegem os visitantes — por exemplo, impedem que o site seja incorporado noutros sites para enganar utilizadores (clickjacking), ou obrigam o browser a usar sempre ligação segura. Esta ferramenta verifica se esses cabeçalhos de proteção existem e estão bem configurados.

Quando usar: Quando queres saber se um site tem as proteções básicas ativas — útil para auditar o teu próprio site ou para um exercício de segurança.

Como usar:

```
python3 tools/check_headers.py https://exemplo.com --lang pt
```

Exemplo de resultado:

URL: `https://exemplo.com`

Estado: 200

✓ Strict-Transport-Security presente

x Content-Security-Policy em falta

Risco: Permite ataques de injeção de conteúdo (XSS)

Correção: Adicionar o cabeçalho com uma política adequada

Pontuação: 6/9

Opções úteis:

- `--json` — resultado em formato JSON (para processar automaticamente)
 - `--no-color` — sem cores (útil para copiar para documentos)
 - `--timeout 20` — esperar mais tempo por sites lentos (padrão: 10 segundos)
-

Verificador de Cookies — `cookie_check.py`

Para que serve: Os cookies são pequenos ficheiros que os sites guardam no teu browser para te manter ligado, lembrar preferências, etc. Se um cookie de sessão (que prova que estás autenticado) não estiver bem protegido, um atacante pode roubá-lo e entrar na tua conta. Esta ferramenta verifica se os cookies do site têm as proteções corretas.

Quando usar: Ao auditar um site onde utilizadores fazem login — especialmente em lojas online, bancos ou qualquer serviço com contas.

Como usar:

```
python3 tools/cookie_check.py https://exemplo.com --lang pt
```

Exemplo de resultado:

Cookie: sessionid

- ✗ HttpOnly: em falta → JavaScript pode ler este cookie
 - ✓ Secure: presente
 - ✗ SameSite: em falta → Vulnerável a ataques CSRF
 - Expiração: 365 dias → Aviso: expiração muito longa
-

Verificador de Partilha entre Sites (CORS) — `cors_check.py`

Para que serve: Imagina que tens sessão iniciada no teu banco online. Se o banco tiver uma configuração CORS mal feita, um site malicioso que visites pode fazer pedidos ao banco em teu nome — sem que te apercebas. Esta ferramenta testa se um site está vulnerável a este tipo de ataque.

Quando usar: Em APIs e sites que partilham dados com outras origens — especialmente quando há autenticação envolvida.

Como usar:

```
python3 tools/cors_check.py https://api.exemplo.com/utilizadores --lang pt
```

Atenção: Usa apenas em sites que tens permissão para testar.

Verificador de Métodos HTTP — `http_methods.py`

Para que serve: Um servidor web pode aceitar vários tipos de pedidos: GET (obter dados), POST (enviar dados), DELETE (apagar), etc. Se um servidor aceitar métodos perigosos que não deveria — como TRACE (pode expor informações) ou DELETE sem autenticação — isso é um problema de segurança. Esta ferramenta descobre que métodos o servidor aceita.

Quando usar: Na fase inicial de análise de um servidor, para perceber a sua superfície de ataque.

Como usar:

```
python3 tools/http_methods.py https://exemplo.com --lang pt
```

Exemplo de resultado:

Métodos testados em https://exemplo.com:

✓ GET	200	
✓ POST	200	
x TRACE	200	← Perigoso: pode expor cabeçalhos internos
x DELETE	405	(bloqueado – bem)

Leitor de robots.txt e Sitemap — `robots_check.py`

Para que serve: O ficheiro robots.txt é como um mapa público que os sites disponibilizam para dizer aos motores de busca o que devem ou não indexar. Ironicamente, as páginas marcadas como “não indexar” são muitas vezes as mais interessantes — painéis de administração, backups, APIs internas. Esta ferramenta lê esse ficheiro e destaca os caminhos mais relevantes.

Quando usar: No início de qualquer análise de um site — é uma das primeiras coisas a verificar.

Como usar:

```
python3 tools/robots_check.py https://exemplo.com --lang pt
```

Exemplo de resultado:

Caminhos interessantes encontrados:

/admin/	← painel de administração?
/backup/	← ficheiros de backup?

/api/v1/	← endpoint de API
/.git/	← repositório Git exposto!

Comparador de Cabeçalhos ao Longo do Tempo — `header_diff.py`

Para que serve: Guarda uma “fotografia” dos cabeçalhos de segurança de um site e, da próxima vez que correr, compara com a anterior. Útil para detetar regressões — por exemplo, após uma atualização do site que desativou acidentalmente uma proteção.

Quando usar: Para monitorizar o teu próprio site ao longo do tempo, ou para acompanhar mudanças num site durante uma auditoria prolongada.

Como usar:

```
# Primeira vez – guardar fotografia
python3 tools/header_diff.py snapshot https://exemplo.com
```

```
# Vezes seguintes – comparar com a fotografia guardada
python3 tools/header_diff.py diff https://exemplo.com
```

Exemplo de resultado:

Diferenças detetadas:

- Content-Security-Policy	(estava presente, agora em falta)
+ X-Frame-Options	(novo cabeçalho adicionado)

Identificador de Tecnologias — `tech_fingerprint.py`

Para que serve: Identifica que tecnologias um site usa — servidor web (Apache, Nginx), linguagem de programação (PHP, Python), sistema de gestão de conteúdo (WordPress, Drupal), bibliotecas JavaScript, CDN, e até a presença de firewalls de aplicação web (WAF). Esta informação é o ponto de partida para perceber possíveis vulnerabilidades específicas dessas tecnologias.

Quando usar: Na fase de reconhecimento, antes de pesquisar vulnerabilidades conhecidas nas tecnologias identificadas.

Como usar:

```
python3 tools/tech_fingerprint.py https://exemplo.com --lang pt
```

Exemplo de resultado:

Tecnologias identificadas:

Servidor:	nginx/1.24.0
Linguagem:	PHP/8.1
CMS:	WordPress 6.4
CDN:	Cloudflare
WAF:	Cloudflare WAF (detetado)

2. DNS e Rede

O DNS (Domain Name System) é como a agenda telefónica da internet — traduz nomes de domínio (como exemplo.com) em endereços IP. Estas ferramentas consultam e analisam essa “agenda”.

Consultor de Registos DNS — `dns_records.py`

Para que serve: Consulta todos os registos DNS de um domínio — endereços IP, servidores de email, servidores de nome, registos de texto (onde se encontram configurações de segurança de email como SPF e DMARC), e mais. Também testa se o servidor DNS aceita transferências de zona (AXFR) — uma configuração incorreta que pode expor todos os subdomínios de uma vez.

Quando usar: Para obter uma visão geral completa da infraestrutura de um domínio, e para verificar se as proteções de email (SPF, DMARC) estão configuradas.

Como usar:

```
python3 tools/dns_records.py exemplo.com --lang pt
```

Exemplo de resultado:

```
exemplo.com – Registos DNS
```

```
A (endereço IPv4):
```

```
93.184.216.34
```

```
MX (servidores de email):
```

```
10 mail.exemplo.com
```

```
SPF:      ✓ presente
```

```
DMARC:    ✗ em falta ← emails falsos podem passar-se por este domínio
```

Descobridor de Subdomínios — `subfinder.py`

Para que serve: Para além do domínio principal, um site pode ter dezenas de subdomínios — mail.exemplo.com, api.exemplo.com, dev.exemplo.com. Estes subdomínios são muitas vezes menos seguros do que o site principal. Esta ferramenta

encontra-os usando dois métodos: pesquisa em certificados SSL públicos e tentativa de nomes comuns.

Quando usar: Para mapear toda a infraestrutura de um domínio antes de uma análise mais detalhada.

Como usar:

```
python3 tools/subfinder.py exemplo.com --lang pt
```

Exemplo de resultado:

Subdomínios encontrados para exemplo.com:

www.exemplo.com	→ 93.184.216.34
api.exemplo.com	→ 93.184.216.50
dev.exemplo.com	→ 10.0.0.1 ← endereço interno exposto?
mail.exemplo.com	→ 93.184.216.60

Consultor de Registo de Domínio (WHOIS) — whois_check.py

Para que serve: O WHOIS é um registo público que guarda informação sobre quem registou um domínio, quando expira, qual o registar, e servidores de nome. Esta ferramenta consulta esse registo e alerta para domínios prestes a expirar (oportunidade de squatting) ou sem DNSSEC (proteção contra envenenamento de DNS).

Quando usar: Para investigar a propriedade de um domínio, verificar datas de expiração, ou como parte do reconhecimento inicial.

Como usar:

```
python3 tools/whois_check.py exemplo.com --lang pt
```

Exemplo de resultado:

Domínio: exemplo.com

Registado por: IANA

Criado em: 1995-08-14

Expira em: 2025-08-13 ← Aviso: expira em 47 dias

DNSSEC: não configurado

Máquina do Tempo para Sites — `wayback_check.py`

Para que serve: O Wayback Machine (archive.org) guarda fotocópias de sites ao longo do tempo. Esta ferramenta consulta esse arquivo para ver versões antigas de um site — útil para encontrar páginas que foram removidas mas podem ainda conter informação sensível no arquivo, ou para acompanhar a evolução de um site.

Quando usar: Para descobrir conteúdo removido, versões antigas de uma API, ou para investigar a história de um site.

Como usar:

```
python3 tools/wayback_check.py https://exemplo.com --lang pt
```

Exemplo de resultado:

Arquivo mais recente: 2024-03-15 14:32

URL: <https://web.archive.org/web/20240315143200/https://exemplo.com>

Cronologia (últimos 5 arquivos):

2024-03-15	2024-01-20	2023-11-08	2023-08-22	2023-05-10
------------	------------	------------	------------	------------

3. Certificados e Ligações Seguras

O cadeado que vês no browser ao visitar um site HTTPS é garantido por um certificado TLS. Estas ferramentas inspecionam esses certificados e as versões do protocolo de segurança que o servidor aceita.

Inspetor de Certificado e TLS — `tls_inspect.py`

Para que serve: Verifica o certificado SSL/TLS de um site — se é válido, quando expira, para que domínios é válido, quem o emitiu, e se a assinatura é forte. Além disso, testa quais as versões do protocolo TLS que o servidor aceita: versões antigas como SSLv3, TLS 1.0 e TLS 1.1 têm vulnerabilidades conhecidas e não deveriam estar ativas.

Quando usar: Para verificar a saúde do certificado de qualquer site HTTPS, ou para confirmar que versões de protocolo inseguras estão desativadas.

Como usar:

```
python3 tools/tls_inspect.py exemplo.com --lang pt
```

Exemplo de resultado:

Certificado de exemplo.com

Emitido por: Let's Encrypt

Válido para: *.exemplo.com, exemplo.com

Expira em: 2024-09-01 (143 dias)

Assinatura: SHA-256 ✓

Versões TLS aceites:

TLS 1.3 ✓

TLS 1.2 ✓

TLS 1.1 ✗ aceite ← Aviso: versão desatualizada

4. Autenticação e Hashes

Ferramentas para analisar mecanismos de autenticação (tokens JWT) e identificar tipos de dados cifrados (hashes).

Analizador de Tokens JWT — `jwt_inspect.py`

Para que serve: Um JWT (JSON Web Token) é um “bilhete” digital que muitos sites usam para provar que estás autenticado. Parece um texto longo com pontos a separar partes. Na realidade, a maior parte da informação está apenas codificada (não cifrada) — qualquer um pode ler o seu conteúdo. Esta ferramenta decodifica o token e alerta para problemas: algoritmo nulo (“alg:none” — token sem assinatura), expirado, ou com campos obrigatórios em falta.

Quando usar: Quando encontras um JWT numa aplicação web e queres perceber o que contém e se tem vulnerabilidades.

Como usar:

```
python3 tools/jwt_inspect.py eyJhbGciOiJIUzI1NiJ9... --lang pt
```

Exemplo de resultado:

Cabeçalho: { "alg": "HS256", "typ": "JWT" }

Conteúdo: { "sub": "user123", "role": "admin", "exp": 1700000000 }

Problemas encontrados:

- ✗ Token expirado (2023-11-14)
 - ✓ Algoritmo: HS256 (válido)
-

Identificador de Hashes — `hashid.py`

Para que serve: Uma hash é uma representação de dados numa forma que não se consegue reverter diretamente (como uma “impressão digital” de uma palavra-passe). Quando encontras um texto longo e estranho numa base de dados ou ficheiro, esta ferramenta tenta identificar que tipo de hash é — MD5, SHA-256, bcrypt, NTLM, etc. — e indica o modo correspondente no Hashcat (ferramenta de quebra de hashes).

Quando usar: Em CTFs e exercícios de segurança quando encontras hashes e precisas saber por onde começar a quebrá-las.

Como usar:

```
python3 tools/hashid.py 5f4dcc3b5aa765d61d8327deb882cf99 --lang pt
```

Exemplo de resultado:

Hash: 5f4dcc3b5aa765d61d8327deb882cf99

Tipo mais provável: MD5 (confiança: alta)

Hashcat mode: 0

Outros possíveis: MD4, NTLM

5. Palavras-passe e Vulnerabilidades

Ferramentas para avaliar palavras-passe, detetar segredos expostos no código, e pesquisar vulnerabilidades conhecidas.

Verificador de Força de Palavra-passe — `password_strength.py`

Para que serve: Avalia a força de uma palavra-passe com base na sua entropia (aleatoriedade matemática) e verifica se já foi encontrada em bases de dados de fugas de dados — usando o serviço “Have I Been Pwned”. A palavra-passe **nunca é enviada** pela internet: apenas os primeiros 5 caracteres da sua hash são enviados (técnica k-anonymity), sendo a comparação feita localmente.

Quando usar: Para verificar se uma palavra-passe que estás a considerar usar é segura, ou em exercícios de consciencialização de segurança.

Como usar:

```
python3 tools/password_strength.py "MinhaPassword123" --lang pt
```

Exemplo de resultado:

Palavra-passe: MinhaPassword123

Entropia: 52.6 bits (razoável – recomendado: >70 bits)

Comprimento: 16 caracteres

HIBP: encontrada em 847 fugas de dados ← Não usar!

Nota: A palavra-passe é avaliada localmente. Nenhuma informação sensível é enviada pela internet.

Detetor de Segredos no Código — `secrets_scan.py`

Para que serve: Procura no código-fonte (e no histórico git) chaves de API, palavras-passe, tokens e outros segredos que possam ter sido acidentalmente guardados. Este é um erro muito comum — um programador guarda uma chave da AWS num ficheiro, faz commit, e mesmo que apague depois, o histórico fica. Esta ferramenta analisa padrões comuns e usa entropia de Shannon para reduzir falsos positivos.

Quando usar: Antes de publicar um repositório, ou para auditar um projeto existente.

Como usar:

```
# Analisar a pasta atual
python3 tools/secrets_scan.py . --lang pt
```

```
# Incluir histórico git
python3 tools/secrets_scan.py . --git-history --lang pt
```

Exemplo de resultado:

Possíveis segredos encontrados:

```
config/settings.py linha 14:
  AWS_SECRET_KEY = "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"
  Tipo: AWS Secret Key | Entropia: 4.8 bits/char
```

Consultor de Vulnerabilidades Conhecidas (CVE) — `cve_lookup.py`

Para que serve: Quando uma vulnerabilidade de segurança é descoberta num software, recebe um identificador público — por exemplo, CVE-2021-44228 (a famosa Log4Shell). Esta ferramenta consulta a base de dados oficial do NIST (NVD) e mostra os detalhes: descrição, gravidade (pontuação CVSS de 0 a 10), e versões afetadas.

Quando usar: Depois de identificar as tecnologias que um site usa (com `tech_fingerprint.py`), pesquisar CVEs conhecidos para essas versões.

Como usar:

```
python3 tools/cve_lookup.py CVE-2021-44228 --lang pt
```

Exemplo de resultado:

CVE-2021-44228 — Log4Shell

Descrição: Execução remota de código em Apache Log4j 2.x antes de 2.15.0

Gravidade: CRÍTICA (CVSS: 10.0 / 10)

Publicado: 2021-12-10

Afeta: Apache Log4j 2.0 até 2.14.1

6. Criptografia e CTF

Ferramentas para decodificar dados e resolver desafios de criptografia, comuns em competições de CTF (Capture The Flag).

Descodificador Automático — `multidecode.py`

Para que serve: Quando encontras um texto estranho e não sabes em que formato está codificado, esta ferramenta tenta automaticamente vários formatos: Base64, Base32, hexadecimal, URL encoding, binário, e ROT13. Com `--cascade` aplica as decodificações em cadeia — útil em CTFs onde os dados são codificados várias vezes.

Quando usar: Sempre que encontras um texto codificado e precisas de o decodificar rapidamente sem ter de adivinhar o formato.

Como usar:

```
# Texto simples
python3 tools/multidecode.py "SGVsbG8gV29ybGQ="

# Texto codificado múltiplas vezes
python3 tools/multidecode.py "SGVsbG8gV29ybGQ=" --cascade
```

Exemplo de resultado:

```
Input:      SGVsbG8gV29ybGQ=
Detetado:   Base64
Resultado:  Hello World
```

Quebrador de Cifra XOR — `xor_crack.py`

Para que serve: O XOR é uma operação matemática simples usada em criptografia básica. Quando um texto é “cifrado” com XOR usando uma chave pequena, é relativamente fácil de decifrar usando análise de frequência (as letras mais comuns em inglês têm distribuições conhecidas). Esta ferramenta tenta descobrir a chave e recuperar o texto original.

Quando usar: Em CTFs quando encontras um texto cifrado com XOR — especialmente em desafios de criptografia básica.

Como usar:

Texto em hexadecimal, chave de 1 byte

```
python3 tools/xor_crack.py --hex "1b37373331363f78151b7f2b783431333d78"
```

Chave de comprimento variável (multi-byte)

```
python3 tools/xor_crack.py --hex "arquivo.hex" --keylen 3
```

Exemplo de resultado:

Chave encontrada: 0x58 ('X')

Texto decifrado: Cooking MC's like a pound of bacon

7. Orquestração

Ferramentas que combinam várias análises numa só execução, gerando um relatório completo.

Reconhecimento Automático Completo — recon.py

Para que serve: Em vez de correr cada ferramenta manualmente, esta corre automaticamente cinco análises em sequência — descoberta de subdomínios, cabeçalhos de segurança, certificado TLS, cookies, e DNS — e gera um relatório único em formato Markdown. É o ponto de partida para uma análise rápida de qualquer domínio.

Quando usar: Como primeiro passo numa auditoria ou CTF — dá uma visão geral em poucos minutos.

Como usar:

```
# Análise completa com relatório
python3 tools/recon.py exemplo.com --lang pt
```

```
# Guardar relatório em ficheiro
python3 tools/recon.py exemplo.com --lang pt --output relatorio.md
```

Exemplo de resultado:

```
## Reconhecimento: exemplo.com

### Subdomínios (4 encontrados)
- www.exemplo.com
- api.exemplo.com

### Cabeçalhos de Segurança
Pontuação: 5/9 | 4 problemas encontrados

### Certificado TLS
Expira em: 2024-09-01 (143 dias) ✓
```

8. HackTheBox

Ferramentas específicas para a plataforma de aprendizagem HackTheBox.

Estatísticas do HackTheBox — `htb_stats.py`

Para que serve: Gera um badge (imagem de estatísticas) para o teu perfil HackTheBox pronto a incorporar num README ou portfolio, e mostra as estatísticas do teu perfil (ranking, máquinas resolvidas, pontos) quando forneces um token de autenticação.

Quando usar: Para atualizar o teu portfolio com as tuas estatísticas HTB mais recentes.

Como usar:

```
# Badge sem autenticação (só markdown do badge)
python3 tools/htb_stats.py --username ciberacaro
```

```
# Estatísticas completas com token
python3 tools/htb_stats.py --username ciberacaro --token SEU_TOKEN_HTB
```

Exemplo de resultado:

Badge Markdown:

```
[![HackTheBox](https://www.hackthebox.eu/badge/...)](https://...)
```

Estatísticas:

Ranking: #4821

Máquinas resolvidas: 12 (7 user + 5 root)

Pontos: 480

9. Portfolio

Ferramentas para gerir o conteúdo do portfolio (writeups).

Gerador de Writeups — `new_writeup.py`

Para que serve: Um writeup é um relatório detalhado de como resolveste um desafio de segurança (CTF, máquina HTB, bug bounty). Esta ferramenta cria automaticamente um ficheiro pronto a editar, com o formato correto para o site (frontmatter Jekyll/Chirpy) e as secções standard de um writeup: reconhecimento, exploração, escalada de privilégios, etc.

Quando usar: Sempre que queres escrever um novo writeup para o portfolio — evita ter de copiar e formatar manualmente.

Como usar:

```
# Criar writeup para uma máquina HTB
python3 tools/new_writeup.py --title "HackTheBox - NomeDaMaquina" --
category htb
```

```
# Criar writeup para um CTF
python3 tools/new_writeup.py --title "CTF 2024 - Web Challenge" --
category web
```

Exemplo de resultado:

Criado: `_posts/2024-03-15-hackthebox-nomedamaquina.md`

Abre o ficheiro criado e preenche as secções. O site atualiza automaticamente quando fizeres push para o GitHub.

Glossário Rápido

Termos técnicos usados neste guia, explicados de forma simples.

Termo	Explicação simples
API	Porta de comunicação entre programas — permite que apps se falem entre si
CORS	Regras que definem se um site pode pedir dados a outro site diferente
Cookie	Pequeno ficheiro guardado no browser com informações de sessão
CTF	Competição de segurança onde se resolvem desafios para obter “bandeiras” (flags)
CVE	Identificador público de uma vulnerabilidade de segurança conhecida
DNS	Sistema que traduz nomes de domínio (exemplo.com) em endereços IP
Entropia	Medida de aleatoriedade — mais alta = mais difícil de adivinhar
Hash	Representação matemática de um dado que não se consegue reverter diretamente
HTTPS	Versão segura do HTTP — a ligação é cifrada com TLS
JWT	Token digital usado para autenticação em aplicações web
TLS/SSL	Protocolo que cifra as comunicações na internet (o cadeado no browser)
WAF	Firewall de aplicação web — filtra pedidos maliciosos antes de chegarem ao servidor
WHOIS	Base de dados pública com informação sobre o registo de domínios
XOR	Operação matemática usada em criptografia básica